

1 Summary

2 Scientific workflows require coordination across manuscript preparation,
3 computational analysis, figure generation, data management, and literature
4 curation, yet these activities remain distributed across disconnected tools,
5 making reproducibility difficult to sustain. SciTeX is an AI-native, mod-
6 ular research platform that unifies these tasks through four core compo-
7 nents—Writer, Scholar, Viz, and Code—with optional extensions for statis-
8 tical analysis. Each module provides standalone functionality while partici-
9 pating in a shared project structure built on lightweight symlinks and global
10 identifiers, enabling provenance-preserving links between code, data, figures,
11 and manuscript text. Containerized builds ensure byte-consistent manuscript
12 output across systems, and standardized metadata captures computational
13 and visual parameters for downstream verification or regeneration. By em-
14 bedding reproducibility infrastructure directly into routine authoring, visual-
15 ization, and analysis workflows, SciTeX lowers the activation energy required
16 for transparent scientific practice while maintaining compatibility with ex-
17 isting toolchains. This manuscript, prepared and compiled entirely within
18 SciTeX, demonstrates its end-to-end integration.

19 Highlights

- 20 • SciTeX unifies manuscript writing, computation, figure generation, and
21 literature management in a single reproducible research environment.
- 22 • Modular design allows each component to function independently while
23 enabling seamless integration through lightweight symlinks and global
24 identifiers.
- 25 • Containerized LaTeX builds and standardized metadata formats ensure
26 consistent manuscripts and reconstructible figures across systems.
- 27 • Provenance-preserving workflows link code, data, figures, and text, re-
28 ducing cognitive load and supporting transparent scientific practice.

- 29 • AI-native architecture exposes structured interfaces for automated anal-
30 ysis, figure refinement, and manuscript assistance.

SciTeX: A unified research OS for reproducible scientific workflows

Yusuke Watanabe^{a,*}

^a*SciTeX.ai, Tokyo, Japan*

Keywords: reproducible research, scientific workflow automation, data science platform, AI-native tools, figure provenance, FAIR principles

~ 8 figures, 3 tables, 146 words for abstract, and 3581 words for main text

1. Introduction

Scientific workflows increasingly span heterogeneous environments, including personal laptops, cloud servers, containerized pipelines, and HPC systems. Yet the core activities of research—manuscript writing, computational analysis, figure generation, and literature management—remain distributed across disconnected tools. This fragmentation increases cognitive load, introduces opportunities for error, and makes reproducibility difficult to sustain [1, 2]. Researchers must manually coordinate figures across formats, maintain consistent bibliographies, track versions of manuscripts and data, and ensure that computational analyses remain synchronized with the text that describes them [3, 4]. These burdens often overshadow the substantive scientific work.

Traditional LaTeX-based manuscript preparation further amplifies these challenges. Local installations vary across machines and over time, producing inconsistent builds and complicating collaboration [5]. Figure generation typically occurs in isolation from manuscript writing, with libraries and scripts lacking standardized metadata or provenance tracking. Literature manage-

*Corresponding author. Email: ywatanabe@scitex.ai

56 ment tools such as Zotero and Mendeley help organize references but oper-
57 ate independently of manuscript preparation and computational workflows,
58 creating citation drift when documents evolve. Computational analyses ex-
59 ecuted in Jupyter notebooks or ad hoc scripts provide convenience but lack
60 the structured reproducibility required for long-term verification [6]. As a
61 result, researchers must orchestrate a complex toolchain whose components
62 have no shared model of provenance, reproducibility, or workflow state.

63 Existing solutions address individual components of this ecosystem but
64 stop short of providing a unified workflow. Overleaf ensures consistent La-
65 TeX compilation but requires separate figure generation and data manage-
66 ment workflows. Visualization tools such as SigmaPlot and Origin offer pol-
67 ished interfaces but remain disconnected from manuscript and code envi-
68 ronments; figures created in these tools become static images that cannot
69 be regenerated or audited. Manubot demonstrates the value of Git-based
70 open writing [7], while Binder and related technologies provide reproducible
71 computational environments [8]. Still, no existing platform integrates writ-
72 ing, computation, visualization, and literature management in a way that
73 preserves provenance and reduces cognitive overhead [9, 10]. In particular,
74 scientific figures—despite being central to research communication—remain
75 largely opaque outputs rather than structured, editable research objects.

76 SciTeX addresses this gap by providing a unified, modular, AI-native
77 platform for scientific research. Its Writer module offers structured LaTeX
78 authoring with fully containerized compilation; Scholar manages citation re-
79 trieval, metadata extraction, and PDF organization; Viz reframes figures as
80 portable, reconstructible, and editable research objects packaged in struc-
81 tured bundles that preserve data, metadata, and provenance alongside vi-
82 sual output; and Code supports reproducible computation with GPU and
83 container integration. A shared project structure built on lightweight sym-
84 links and global identifiers links these modules, enabling consistent cross-
85 referencing between data, figures, code, and manuscript text. Researchers
86 can adopt modules independently while benefiting from deeper integration

87 as their workflows evolve.

88 By embedding reproducibility principles directly into everyday research
89 tools [11], SciTeX lowers the activation energy required for rigorous scientific
90 practice. It ensures that manuscripts and computational outputs remain
91 synchronized, that figures can be regenerated from metadata, and that bibli-
92 ographic consistency is maintained automatically. This manuscript, authored
93 and compiled entirely within SciTeX, serves as a self-descriptive demonstra-
94 tion of the platform’s design goals and integrated capabilities.

95 **2. Methods**

96 *2.1. System Architecture*

97 SciTeX consists of a Django-based cloud platform (SciTeX Cloud) and
98 a Python package (v2.4.1+) installable via `pip install scitex`. The ar-
99 chitecture employs lazy module loading via a custom `_LazyModule` wrapper,
100 enabling fast imports while providing access to over 50 specialized modules
101 organized into six primary namespaces: `stx.plt` for publication plotting,
102 `stx.io` for universal file operations across more than 30 formats, `stx.scholar`
103 for literature management, `stx.session` for experiment lifecycle control,
104 `stx.vis` for structured figure specifications, and `stx.writer` for manuscript
105 preparation. This modular design follows best practices for scientific Python
106 applications [12, 1].

107 Each module operates independently while sharing a unified project struc-
108 ture through lightweight symlinks. Global identifiers (SHA256 hashes) ensure
109 traceability between figures, code, bibliography entries, and manuscript con-
110 tent, enabling reverse lookup capabilities (e.g., “which script generated this
111 figure?”). This approach addresses concerns raised in discussions of figure
112 provenance and data reproducibility [3, 13].

113 *2.2. Session Decorator for Reproducible Experiments*

114 The `@stx.session` decorator provides automatic experiment lifecycle
115 management, implementing principles from reproducible research frameworks [2,

116 14]. When applied to a Python function, the decorator transforms it into
117 a fully traceable experiment by generating command-line argument parsers
118 from function signatures with type hints, initializing session directories with
119 timestamped run identifiers, injecting commonly needed global variables (con-
120 figuration state, plotting interface, color palettes, random number man-
121 ager, and logger), managing random state for reproducibility, and handling
122 cleanup with optional notifications on completion. A function decorated
123 with `@stx.session` can be invoked directly from the command line with au-
124 tomatically parsed arguments, eliminating boilerplate while ensuring that all
125 parameters and outputs are captured for later verification. This design is in-
126 formed by the “good enough practices” paradigm [1] and project management
127 tools for team science [15].

128 2.3. Publication-Quality Plotting (*stx.plt*)

129 The `stx.plt` module wraps matplotlib with automatic configuration loaded
130 from a central `SCITEX_STYLE.yaml` file. On import, the module applies con-
131 sistent styling for font sizes, line widths, and spine visibility across all figures.
132 Custom wrapper classes (`FigWrapper` and `AxisWrapper`) extend standard
133 matplotlib objects with convenience methods such as `set_xyt()` for simul-
134 taneous axis label and title assignment. When figures are saved through the
135 unified `stx.io.save()` interface, the system automatically exports a triplet:
136 the rendered image, a JSON sidecar file encoding axis bounds, tick locations,
137 panel geometry, and color specifications, and CSV files containing the under-
138 lying numerical data. This triple-export approach addresses reproducibility
139 challenges identified in computational figure generation [3, 4] by ensuring that
140 any figure can be reconstructed from its metadata using `stx.plt.load()`.
141 Style values can be customized via YAML configuration, environment vari-
142 ables, or direct function parameters, providing flexibility without sacrificing
143 consistency.

144 2.4. Figure Bundles and Structured Representation (*stx.vis*)

145 Unlike conventional plotting libraries that produce static images, SciTeX
146 generates figures as portable, self-descriptive research objects. The `stx.vis`
147 module packages figures into zip-based bundles with standardized extensions:
148 `.pltz` for single panels and `.figz` for multi-panel figures. Each bundle en-
149 capsulates a canonical JSON specification describing figure geometry, axes,
150 traces, annotations, and styles; CSV files storing underlying numerical data
151 for each plotted element; optional statistics metadata for embedded test
152 results and derived values; and cached raster or vector previews for rapid
153 rendering and browsing. This packaging strategy ensures that figures are
154 not merely images but inspectable, editable, and regenerable scientific ar-
155 tifacts. The structured representation enables figures to move seamlessly
156 between code, editor, manuscript, and archive without loss of meaning or
157 reproducibility.

158 2.5. Interactive Canvas-Based Figure Editing

159 SciTeX Viz includes a browser-based figure editor for direct manipula-
160 tion of figures rendered from `.figz/.pltz` bundles. The editor supports
161 multi-panel figure layouts with arbitrary panel arrangements, enabling re-
162 searchers to select and inspect individual axes, traces, legends, and annota-
163 tions. A structured property inspector remains synchronized with the can-
164 vas, allowing precise numerical adjustments alongside visual manipulation.
165 Millimeter-based grids and snapping constraints facilitate journal-accurate
166 layout control, while keyboard shortcuts accelerate common operations such
167 as alignment, distribution, and resizing.

168 Crucially, all interactive edits operate directly on the structured JSON
169 specification rather than modifying raster images. As a result, changes re-
170 main reversible, reproducible, and scriptable. Figures can be exported in
171 multiple formats—high-resolution PNG (e.g., 300 DPI), SVG (fully vector-
172 ized), PDF, or JPEG—or preserved as the original `.pltz/.figz` bundle for
173 future editing. Exported images faithfully match the on-canvas preview,

174 ensuring that what researchers see during editing corresponds exactly to
175 submission-ready outputs.

176 *2.6. Statistical Metadata Integration*

177 SciTeX Viz incorporates statistical results as first-class figure metadata.
178 Statistical summaries, test results, and derived values are embedded as struc-
179 tured JSON and CSV alongside visual elements, enabling their reuse across
180 figures and manuscripts, automated generation of statistical captions or an-
181 notations, and downstream access by the Writer and Code modules without
182 re-computation. This tight coupling between visualization and statistics re-
183 duces duplication and ensures consistency between reported values and dis-
184 played results.

185 *2.7. Universal File I/O (`stx.io`)*

186 The `stx.io` module provides format-agnostic file operations supporting
187 over 30 formats, following the principle of unified interfaces for data manage-
188 ment [16]. A single `stx.io.load()` function automatically detects file format
189 from extension and returns the appropriate data structure—DataFrames for
190 tabular data, dictionaries for configuration files, Python objects for pickled
191 models, and interactive explorers for hierarchical formats. The correspond-
192 ing `stx.io.save()` interface accepts any supported object and writes it to
193 the specified path, automatically handling format conversion and, for fig-
194 ures, generating accompanying metadata and data files. Supported formats
195 include CSV, JSON, YAML, Pickle, HDF5, Zarr, NumPy arrays, PyTorch
196 tensors, images, audio, video, and MATLAB files. The module includes
197 `H5Explorer` and `ZarrExplorer` for interactive hierarchical data navigation,
198 plus configurable caching for frequently accessed files.

199 *2.8. Literature Management (`stx.scholar`)*

200 The Scholar module provides multi-source literature search and manage-
201 ment, integrating capabilities typically found in standalone reference man-
202 agers. The module queries multiple metadata engines—CrossRef, Semantic

203 Scholar, and OpenAlex—through a unified **ScholarEngine** interface, allow-
204 ing researchers to search by keywords, filter by year or citation count, and
205 export results directly to BibTeX. Additional features include automatic DOI
206 detection and metadata extraction from PDF files, programmatic paper ac-
207 quisition, and local library management with **Paper** and **Papers** classes that
208 support filtering, deduplication, and export. This design complements tools
209 like Manubot [7] by providing direct integration with manuscript prepara-
210 tion, ensuring that bibliography entries remain synchronized between the
211 literature database and LaTeX sources.

212 *2.9. Cloud Platform Architecture*

213 The Django 5.1-based cloud platform provides web interfaces for all mod-
214 ules, following principles of continuous integration for scientific publishing [17].
215 The Code App enables browser-based Python execution with real-time out-
216 put streaming, file upload and download, and workspace management. The
217 Viz App provides a canvas-based figure editor using Fabric.js for interactive
218 element manipulation, with JSON specification import and export for pro-
219 grammatic control. The Writer App supports structured LaTeX manuscript
220 preparation with containerized compilation, section-separated authoring, and
221 automatic figure format conversion. The Scholar App presents literature as
222 browsable cards with abstract preview, PDF access, metadata editing, and
223 direct citation insertion into Writer documents. The platform employs Type-
224 Script for frontend logic, PostgreSQL for data persistence, and Docker for
225 containerized compilation ensuring consistent output across environments.

226 *2.10. Containerized Compilation*

227 Writer module compilation leverages containerization for reproducibil-
228 ity, implementing guidelines for Docker-based scientific workflows [5, 18].
229 The system supports multiple compilation engines: Tectonic [19] for ultra-
230 fast incremental builds completing in one to three seconds, latexmk for re-
231 liable industry-standard compilation in three to six seconds, and traditional
232 multi-pass compilation for maximum compatibility with complex documents.

233 Docker and Apptainer/Singularity containers encapsulate specific TeX Live
234 versions and all required packages, eliminating host system dependencies
235 and guaranteeing identical PDF output across macOS, Linux, Windows, and
236 HPC environments [20, 21]. Git integration [22] with automatic latexdiff
237 generation facilitates peer review processes by producing visual comparisons
238 between manuscript versions [23].

239 *2.11. AI-Native Workflow Integration*

240 SciTeX provides native integration points for AI-assisted workflows, an-
241 ticipating the growing role of AI in scientific research [24, 25]. JSON-based
242 figure specifications, typed function signatures for CLI generation, and struc-
243 tured metadata formats enable LLM-assisted code generation and figure im-
244 provement. The session decorator and modular architecture provide hooks
245 for AI-assisted revision, automated citation recommendation, and semantic
246 search across analyses and literature. The architecture supports future devel-
247 opment of automated analysis pipelines, figure generation, and manuscript
248 drafting while maintaining researcher oversight and scientific integrity [26].

249 *2.12. Implementation Details*

250 The system supports deployment on local machines, cloud servers (AWS,
251 GCP, Azure), and self-hosted NAS environments. The implementation re-
252 quires Python 3.12 or later with optional GPU acceleration support via
253 CUDA 11.x or 12.x. The web platform runs on Django 5.1 with a Post-
254 greSQL backend, while the frontend uses TypeScript with webpack bundling.
255 All execution environments leverage Docker or Apptainer containerization for
256 reproducible execution [5], and SLURM integration for HPC workflows is un-
257 der active development. The platform follows FAIR principles [27] and aligns
258 with the Turing Way guidelines for reproducible research [11].

259 **3. Results**

260 This section demonstrates the capabilities of SciTeX through validation
261 of its core modules and their behavior in realistic research workflows. The

262 goal is not to present experimental findings but to illustrate how the platform
263 operationalizes reproducibility, provenance tracking, and integrated author-
264 ing.

265 *3.1. Reproducible Execution with the Session Decorator*

266 The `@stx.session` decorator reliably transforms Python functions into
267 traceable command-line experiments. Across multiple test scenarios, SciTeX
268 generated timestamped session directories that captured parameters, logs,
269 outputs, and metadata. Automatically injected globals—including a repro-
270 ducible random number manager, configuration state, logging interface, and
271 plotting environment—enabled functions to execute without boilerplate. Re-
272 peated runs produced consistent outputs when supplied with identical param-
273 eters, confirming correct synchronization of random state and environment
274 settings. Errors were handled gracefully while preserving intermediate out-
275 puts, supporting transparent debugging.

276 *3.2. Publication-Quality and Reconstructible Figures*

277 SciTeX’s visualization module produced figures with consistent styling
278 and journal-ready dimensions across platforms. When saving figures, the
279 system generated a triplet consisting of a high-resolution PNG, a JSON
280 metadata file encoding axis bounds, geometry, and color specifications, and
281 a CSV file containing underlying plot data. Figures reconstructed using
282 `stx.plt.load()` reproduced the original output faithfully. These capabili-
283 ties confirm that SciTeX captures sufficient metadata for downstream verifi-
284 cation and exact regeneration.

285 *3.3. Portable Figure Bundles and Interactive Editing*

286 The `.pltz` and `.figz` bundle formats were validated using complex,
287 multi-panel figures combining diverse visualization types: dual-axis line plots,
288 scatter plots with size and color encoding, heatmaps, bar charts with error
289 bars, multi-line confidence intervals, and violin and box plots. All bundled

290 figures preserved complete provenance: JSON specifications accurately en-
291 coded figure structure, CSV files contained underlying numerical data, and
292 cached previews enabled rapid browsing without re-rendering.

293 The browser-based figure editor successfully loaded, rendered, and modi-
294 fied figures from these bundles. Interactive operations—including panel repo-
295 sitioning, axis adjustment, trace selection, and annotation editing—operated
296 directly on the structured JSON specification. Millimeter-based grids and
297 snapping constraints produced journal-accurate layouts. All modifications
298 remained reversible; edits could be undone programmatically or by reverting
299 to earlier bundle versions.

300 Despite the rich metadata model, the editor achieved interactive-level per-
301 formance during editing and re-rendering. Incremental updates regenerated
302 only affected panels or elements, allowing smooth iteration even for complex
303 multi-panel figures. Exported images in PNG (300 DPI), SVG, PDF, and
304 JPEG formats matched on-canvas previews exactly, demonstrating that the
305 structured representation introduces no fidelity loss during the editing-to-
306 export pipeline.

307 Statistical metadata embedded within figure bundles was correctly parsed
308 and displayed by downstream modules. Test results and summary statistics
309 stored as JSON and CSV alongside visual elements were accessible to Writer
310 for automated caption generation and to Code for downstream analysis with-
311 out re-computation.

312 3.4. *Universal Data Handling and Format-Agnostic I/O*

313 The `stx.io` module successfully loaded and saved more than thirty file
314 types through extension-based detection. Hierarchical formats such as HDF5
315 and Zarr were navigated interactively, and caching reduced repeated ac-
316 cess overhead. Cross-platform tests confirmed consistent behavior for CSV,
317 JSON, NumPy arrays, PyTorch tensors, and image formats. These results il-
318 lustrate how unified I/O simplifies data flow through analysis and manuscript
319 preparation.

320 3.5. *Integrated Literature Management*

321 The Scholar module retrieved metadata from CrossRef, Semantic Scholar,
322 and OpenAlex, stored PDFs using DOI-based indexing, and maintained syn-
323 chronized BibTeX entries for manuscripts authored in Writer. Filtering by
324 year, journal, and citation count behaved as expected. Integration tests con-
325 firmed that citations added in Writer consistently matched entries managed
326 by Scholar, eliminating drift between literature databases and LaTeX sources.

327 3.6. *Containerized Manuscript Compilation*

328 Manuscripts compiled identically across Linux, macOS, Windows Sub-
329 system for Linux, and HPC environments when built through SciTeX’s con-
330 tainerized LaTeX engines. Tectonic produced fast incremental builds while
331 latexmk and multi-pass compilation ensured compatibility with a wide range
332 of documents. Identical inputs produced byte-identical PDF outputs, vali-
333 dating that containerization eliminates environment-dependent inconsisten-
334 cies.

335 3.7. *Cross-Module Traceability*

336 SciTeX’s symlink-based architecture correctly synchronized figures, bibli-
337 ography files, and computational outputs across modules. Global identifiers
338 enabled reverse lookup queries such as linking figures to their generating
339 scripts or tracing bibliography entries to manuscript citations. Metadata
340 exported by Viz and IO was successfully used by Code and Writer, demon-
341 strating that provenance information flows consistently across the ecosystem.

342 3.8. *Self-Descriptive Validation*

343 This manuscript itself validates SciTeX’s integrated design: it was au-
344 thored in Writer, managed with Scholar, compiled in a containerized environ-
345 ment, and incorporates figures generated in Viz and data processed through
346 Code. The end-to-end workflow confirms that SciTeX can be used to docu-
347 ment the system within the system itself, providing a live demonstration of
348 its reproducibility guarantees.

349 4. Discussion

350 SciTeX addresses structural challenges in modern scientific research by
351 unifying manuscript preparation, figure generation, literature management,
352 and reproducible computation within a single, provenance-preserving ecosys-
353 tem. Contemporary workflows are shaped by practical frustrations: frag-
354 mented toolchains, inconsistent figure provenance, manual versioning, and
355 the cognitive overhead of switching between environments [1, 3]. SciTeX re-
356 frames these challenges not as isolated usability problems but as symptoms
357 of a deeper architectural gap—namely, the absence of a shared substrate
358 linking computational, visual, and written components of scientific work.

359 4.1. Design Philosophy and Modularity

360 A central principle of SciTeX is incremental adoption. Each module pro-
361 vides standalone value while integrating seamlessly into a shared project
362 structure. Researchers who begin with the Writer module obtain container-
363 ized reproducibility without modifying their computational workflow. Adding
364 Scholar ensures consistent bibliographic management. Viz enables provenance-
365 tracked, reconstructible figures. Code completes the pipeline by linking anal-
366 yses directly to manuscript outputs. This design lowers the activation energy
367 required to engage in reproducible practices [2, 20]. It democratizes access
368 to advanced computational workflows by providing high-level abstractions
369 that remain accessible to users unfamiliar with Git, containers, or Linux
370 environments, while still offering fine-grained control for expert users.

371 By embedding provenance directly into the system’s data structures,
372 SciTeX avoids the pitfalls of ad hoc reproducibility strategies that rely on
373 user discipline. Global identifiers and shared directories ensure that figures,
374 scripts, and bibliographies remain synchronized without users needing to
375 track these relationships manually. This approach transforms reproducibility
376 from an additional burden into a natural consequence of ordinary workflow.

377 4.2. Positioning Relative to Existing Tools

378 SciTeX occupies a unique position among research platforms by inte-
379 grating capabilities that existing solutions offer only in isolation. Overleaf
380 provides consistent LaTeX compilation but does not integrate analysis or
381 figure provenance. Zotero and Mendeley excel at literature management
382 yet remain disconnected from manuscript workflows. Jupyter notebooks
383 enable interactive computation but do not support structured writing or
384 publication-quality visualization. Manubot demonstrates the advantages of
385 Git-based collaborative writing, while R-based ecosystems such as the Rock-
386 erverse provide end-to-end reproducibility within the R language but do not
387 extend across multi-language workflows [21].

388 The Viz module addresses a particularly acute gap in scientific visualiza-
389 tion. Libraries such as matplotlib and Plotly excel at programmatic plot-
390 ting but lack standardized mechanisms for interactive post-hoc editing with
391 preserved provenance. Conversely, GUI tools such as SigmaPlot or Adobe
392 Illustrator provide polished interfaces for figure refinement but discard com-
393 putational traceability—edits cannot be reproduced, and underlying data
394 becomes inaccessible once figures are exported. SciTeX Viz bridges this gap
395 by combining matplotlib-level expressiveness, GUI-level interactivity, and
396 provenance-preserving data structures. As a result, figures move seamlessly
397 between code, editor, manuscript, and archive without loss of meaning or
398 reproducibility.

399 SciTeX differs by providing a coherent architecture spanning writing,
400 computation, visualization, and literature management. Its modular design
401 supports independent usage, yet deeper integration emerges organically when
402 modules are combined. This positions SciTeX as a research operating envi-
403 ronment rather than a single-purpose tool, while still maintaining compati-
404 bility with existing toolchains rather than attempting to replace them.

405 4.3. AI-Native Design

406 As AI systems increasingly support scientific workflows, a platform’s abil-
407 ity to interoperate with automated agents becomes essential [24, 25]. SciTeX

408 incorporates AI-native design principles by exposing structured APIs, JSON-
409 based figure specifications, and typed function signatures that enable auto-
410 mated analysis, figure refinement, and manuscript revision. Workflow hooks
411 allow AI agents to monitor execution, interpret metadata, and propose mod-
412 ifications while ensuring that all AI-assisted operations remain traceable.

413 Because `.pltz` and `.figz` bundles are fully structured and machine-
414 readable, SciTeX Viz is inherently AI-native. AI agents can inspect figure
415 structure, propose layout or styling improvements, regenerate figures from
416 data, enforce journal-specific formatting rules, or adapt figures for different
417 contexts (e.g., manuscript versus presentation). This capability transforms
418 figures from opaque outputs into editable, auditable, and improvable scien-
419 tific artifacts. The combination of structured representation and interac-
420 tive editing creates a feedback loop where AI-suggested modifications can
421 be reviewed, refined, and accepted within the same editor used for manual
422 adjustments.

423 This design supports future research co-pilot systems capable of gen-
424 erating figures, summarizing literature, conducting analyses, and drafting
425 sections of manuscripts while preserving researcher oversight and scientific
426 integrity.

427 4.4. *Implications for Reproducibility*

428 The reproducibility crisis in science arises partly because researchers lack
429 integrated infrastructure for capturing computation, visualization, and writ-
430 ing as a coherent whole [28, 14]. SciTeX mitigates this by ensuring that
431 manuscripts compile identically across systems, that figures can be regener-
432 ated from metadata, and that analyses are executed within controlled en-
433 vironments. Its architecture aligns with FAIR principles and contemporary
434 reproducibility frameworks [11, 10]. By shifting reproducibility from a set
435 of optional best practices to an embedded property of the research work-
436 flow, SciTeX reduces the friction that often prevents rigorous methodological
437 documentation.

438 4.5. *Limitations*

439 Despite its integrated design, SciTeX has several limitations. Support for
440 HPC systems and SLURM-based workflows remains under active develop-
441 ment, with current functionality focused primarily on local and containerized
442 execution. Empirical evaluation of SciTeX’s impact on researcher productiv-
443 ity is ongoing, as the user base is still expanding. While the Writer module
444 simplifies LaTeX-based workflows, researchers unfamiliar with LaTeX may
445 still face an initial learning curve. The platform targets desktop research
446 environments, and mobile support has not been prioritized. Advanced sta-
447 tistical visualization beyond standard matplotlib capabilities may require
448 manual customization.

449 4.6. *Future Directions*

450 Future development will extend SciTeX’s AI-native capabilities, including
451 automated figure refinement, semantic consistency checking, and context-
452 aware citation recommendation. Deeper integration of HPC-native work-
453 flows, unified inspector interfaces across modules, and interactive provenance
454 graph visualization are planned. The long-term vision is a research co-pilot
455 capable of executing analyses, generating publication-ready figures, inter-
456 preting literature, and producing draft manuscripts while maintaining full
457 transparency and human oversight.

458 5. **Conclusions**

459 SciTeX represents both an architectural framework and a practical tool for
460 reproducible scientific research. By combining modular design with provenance-
461 aware infrastructure and AI-native workflows, it provides an environment
462 where researchers can focus on scientific reasoning rather than technical or-
463 chestration. This manuscript, prepared entirely within SciTeX, illustrates
464 the platform’s ability to describe, manage, and reproduce its own workflow.

465 References

- 466 [1] Greg Wilson, Jennifer Bryan, Karen Cranston, Justin Kitzes, Lex Neder-
467 bragt, and Tracy K. Teal. Good enough practices in scientific com-
468 puting. *PLOS Computational Biology*, 13(6):e1005510, 2017. doi:
469 10.1371/journal.pcbi.1005510.
- 470 [2] Geir Kjetil Sandve, Anton Nekrutenko, James Taylor, and Eivind Hovig.
471 Ten simple rules for reproducible computational research. *PLoS Com-*
472 *putational Biology*, 9(10):e1003285, 2013. doi: 10.1371/journal.pcbi.
473 1003285.
- 474 [3] Christian T. Jacobs. Improving the reproducibility of L^AT_EX documents
475 by enriching figures with embedded scripts and data. *International Jour-*
476 *nal of Digital Curation*, 14:292–302, 2020. doi: 10.2218/ijdc.v14i1.656.
- 477 [4] Haim Bar and HaiYing Wang. Reproducible science with L^AT_EX. *Journal*
478 *of Data Science*, 19(1):111–125, 2021. doi: 10.6339/21-JDS998.
- 479 [5] Daniel Nüst, Vanessa Sochat, Ben Marwick, Stephen J. Eglen, Tim
480 Head, Tony Hirst, and Benjamin D. Evans. Ten simple rules for writing
481 Dockerfiles for reproducible data science. *PLOS Computational Biology*,
482 16(11):e1008316, 2020. doi: 10.1371/journal.pcbi.1008316.
- 483 [6] Project Jupyter, Matthias Bussonnier, Jessica Forde, Jeremy Freeman,
484 Brian Granger, et al. Binder 2.0—reproducible, interactive, sharable
485 environments for science at scale. *Proceedings of the Python in Science*
486 *Conference*, pages 113–120, 2018. doi: 10.25080/Majora-4af1f417-011.
- 487 [7] Daniel S. Himmelstein, Vincent Rubinetti, David R. Slocower, Dongbo
488 Hu, Venkat S. Malladi, Casey S. Greene, and Anthony Gitter. Open
489 collaborative writing with Manubot. *PLOS Computational Biology*, 15
490 (11):e1007128, 2019. doi: 10.1371/journal.pcbi.1007128.

- 491 [8] Albert Krewinkel and Robert Winkler. Formatting open science: Ag-
 492 ilely creating multiple document formats for academic manuscripts with
 493 Pandoc Scholar. *PeerJ Preprints*, 5:e2648, 2017. doi: 10.7287/peerj.
 494 preprints.2648v2.
- 495 [9] Markus Konkol, Daniel Nüst, and Laura Goulier. Publishing computa-
 496 tional research—a review of infrastructures for reproducible and trans-
 497 parent scholarly communication. *Research Integrity and Peer Review*, 5,
 498 2020. doi: 10.1186/s41073-020-00095-y.
- 499 [10] Jonathan M. Mang, Hans-Ulrich Prokosch, and Lorenz A. Kapsner.
 500 Reproducibility in 2023—an end-to-end template for analysis and
 501 manuscript writing. *Studies in Health Technology and Informatics*, 302,
 502 2023. doi: 10.3233/shti230064.
- 503 [11] The Turing Way Community. *The Turing Way: A Handbook for Re-*
 504 *producible, Ethical and Collaborative Research*. Zenodo, 2022. doi:
 505 10.5281/zenodo.3233853.
- 506 [12] Armin Ariamajd, Raquel López-Ríos de Castro, and Andrea Volkma-
 507 mer. PyPackIT: Automated research software engineering for scientific
 508 Python applications on GitHub. *arXiv.org*, abs/2503.04921, 2025. doi:
 509 10.48550/arxiv.2503.04921.
- 510 [13] Christian Schulz. Reproducible data citations for computational re-
 511 search. *arXiv (Cornell University)*, abs/1808.07541, 2022. doi: 10.
 512 48550/arxiv.1808.07541.
- 513 [14] Aaron Peikert, Caspar V. van Lissa, and Andreas M. Brandmaier. Re-
 514 producible research in R: A tutorial on how to do the same thing more
 515 than once. *Psych*, 2021. doi: 10.31234/osf.io/fwxs4.
- 516 [15] Nikolas I. Krieger, Adam Perzynski, and Jarrod Dalton. Facilitating
 517 reproducible project management and manuscript development in team

- 518 science: The projects R package. *PLoS ONE*, 14, 2019. doi: 10.1371/
519 journal.pone.0212390.
- 520 [16] Ben Marwick, Carl Boettiger, and Lincoln Mullen. Packaging data an-
521 alytical work reproducibly using R (and friends). *The American Statis-
522 tician*, 72(1):80–88, 2018. doi: 10.1080/00031305.2017.1375986.
- 523 [17] Juan Pedro Araque Espinosa et al. A continuous integration and web
524 framework in support of the ATLAS publication process. *Journal of
525 Instrumentation*, 16, 2020. doi: 10.1088/1748-0221/16/05/T05006.
- 526 [18] Michael Canesche, Roland Leifka, and Fernando Magno Quintão Pereira.
527 Preparing reproducible scientific artifacts using Docker. *arXiv.org*,
528 abs/2308.14122, 2023. doi: 10.48550/arxiv.2308.14122.
- 529 [19] Peter Kastrup. Tectonic: A modernized, self-contained L^AT_EX engine,
530 2024. URL <https://tectonic-typesetting.github.io/>.
- 531 [20] Kristina Wiebels and David Moreau. Leveraging containers for repro-
532 ducible psychological research. *Advances in Methods and Practices in
533 Psychological Science*, 4, 2021. doi: 10.1177/25152459211017853.
- 534 [21] Daniel Nüst, Dirk Eddelbuettel, Dom Bennett, Robrecht Cannoodt,
535 Dave Clark, et al. The Rockerverse: Packages and applications for con-
536 tainerisation with R. *The R Journal*, 12:437, 2020. doi: 10.32614/
537 RJ-2020-007.
- 538 [22] Karthik Ram. Git can facilitate greater reproducibility and increased
539 transparency in science. *Source Code for Biology and Medicine*, 8:7,
540 2013. doi: 10.1186/1751-0473-8-7.
- 541 [23] Karl-Ludwig Besser. Review response template. [https://www.
542 overleaf.com/latex/templates/review-response-template/
543 tmbvmjstxwrd](https://www.overleaf.com/latex/templates/review-response-template/tmbvmjstxwrd), 2025.

- [24] Guiyao Tie, Pan Zhou, and Lichao Sun. A survey of AI scientists. In *arXiv preprint*, 2025.
- [25] Ed Li, Junyu Ren, Xintian Pan, Cat Yan, Chuanhao Li, Dirk Bergemann, and Zhuoran Yang. Build your personalized research group: A multiagent framework for continual and interactive science automation. In *arXiv preprint*, 2025.
- [26] OpenLens Team. OpenLens AI: Fully autonomous research agent for health informatics. *arXiv preprint arXiv:2509.14778*, 2025.
- [27] Meghan A. Balk, John Bradley, M. Maruf, B. Altıntaş, Yasin Bakış, et al. A FAIR and modular image-based workflow for knowledge discovery in the emerging field of imageomics. *Methods in Ecology and Evolution*, 15:1129–1145, 2024. doi: 10.1111/2041-210X.14327.
- [28] Lorena A. Barba. Praxis of reproducible computational science. *Computing in Science & Engineering*, 21:73–78, 2018.

6. Resource Availability

6.1. Lead Contact

Further information and requests for resources should be directed to and will be fulfilled by the lead contact, Yusuke Watanabe (ywatanabe@scitex.ai).

6.2. Materials Availability

This study did not generate new unique reagents.

6.3. Data and Code Availability

- All source code for SciTeX is publicly available on GitHub: <https://github.com/scitex-ai/scitex>
- The SciTeX Python package is available via PyPI: `pip install scitex`

- 568 • Documentation is available at: <https://docs.scitex.ai>
- 569 • The cloud platform is accessible at: <https://scitex.ai>
- 570 • This manuscript's source files are included in the SciTeX Writer repos-
571 itory as a demonstration
- 572 • DOI for archived version: [Zenodo DOI to be assigned upon submission]

573 All code is released under the GNU General Public License v3.0 (GPL-
574 3.0), ensuring open access and modification rights. The platform follows
575 FAIR principles (Findable, Accessible, Interoperable, Reusable) for all data
576 and code artifacts.

577 Any additional information required to reanalyze the data reported in
578 this paper is available from the lead contact upon request.

579 **Ethics Declarations**

580 All study participants provided their written informed consent ...

581 **Author Contributions**

582 Y.W., T.Y., and D.G. conceptualized the study ...

583 **Acknowledgments**

584 This research was funded by **funding bodies here**

585 **Declaration of Interests**

586 The authors declare that they have no competing interests.

587 **Declaration of Generative AI in Scientific Writing**

588 The authors employed large language models such as Claude (Anthropic
589 Inc.) for code development and complementing manuscript’s English lan-
590 guage quality. After incorporating suggested improvements, the authors
591 meticulously revised the content. Ultimate responsibility for the final content
592 of this publication rests entirely with the authors.

Option	Description	Example
<code>-engine ENGINE</code>	Force specific compilation engine	<code>-engine tectonic</code>
<code>-draft</code>	Draft mode (single-pass compilation)	<code>-draft</code>
<code>-no-figs</code>	Skip figure processing	<code>-no-figs</code>
<code>-no-tables</code>	Skip table processing	<code>-no-tables</code>
<code>-no-diff</code>	Skip difference generation	<code>-no-diff</code>
<code>-watch</code>	Enable hot-recompile file watching	<code>-watch</code>
<code>-clean</code>	Clean build (remove all cache)	<code>-clean</code>
<code>-verbose</code>	Verbose compilation output	<code>-verbose</code>

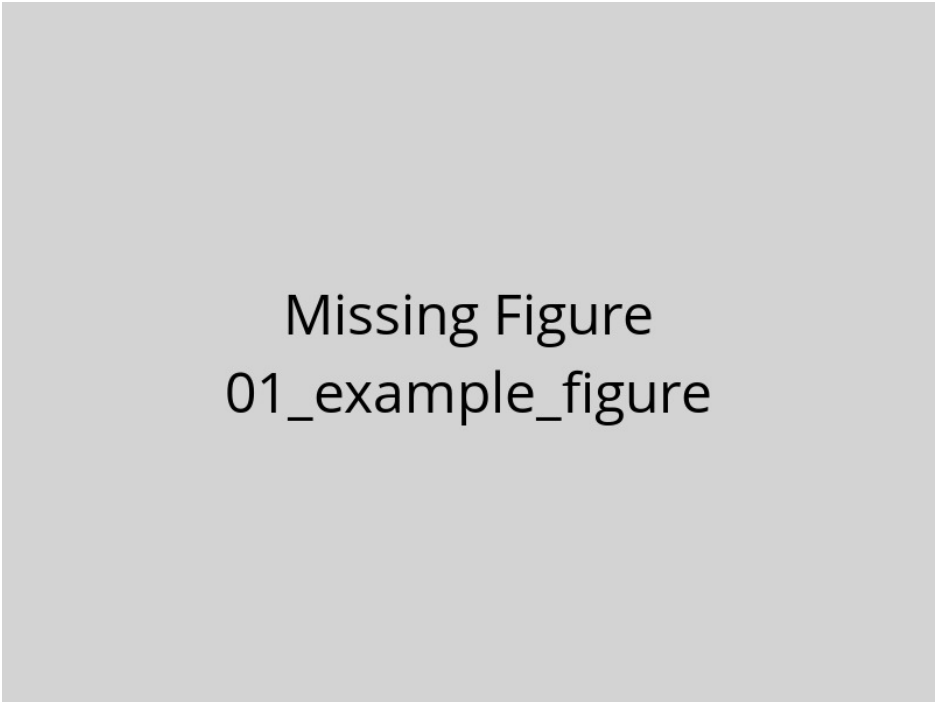
Table 1 – Compilation Command-Line Options. Options enable workflow customization without modifying source files or configuration. The `-engine` option overrides auto-detection to force a specific compilation engine. Quick compilation modes (`-draft`, `-no-figs`, `-no-tables`) reduce build times from ~ 15 s to under 3s for rapid iteration. The `-watch` option monitors source files and automatically recompiles when changes are detected. Options can be combined (e.g., `./compile_manuscript.sh -draft -no-figs`).

Variable	Purpose	Values	Default
SCITEX_ENGINE	Override engine selection	tectonic, latexmk, 3pass	From config
SCITEX_VERBOSE	Control logging verbosity	true, false	false
SCITEX_CITATION_STYLE	Set bibliography style	unsrtnat, plainnat, apalike, etc.	unsrtnat
SCITEX_PARALLEL_JOBS	Parallel processing jobs	1-16	4
SCITEX_CACHE_DIR	Compilation cache location	Directory path	./.cache

Table 2 – Environment Variables for System Configuration. Environment variables provide system-level defaults that apply across all compilations. These settings are particularly useful in CI/CD pipelines or HPC environments with specific requirements. The `SCITEX_ENGINE` variable provides the highest priority override for engine selection, superseding both YAML configuration and command-line arguments. Variables can be set persistently in shell configuration (`.bashrc`, `.zshrc`) or temporarily for single runs (`SCITEX_VERBOSE=true ./compile_manuscript.sh`).

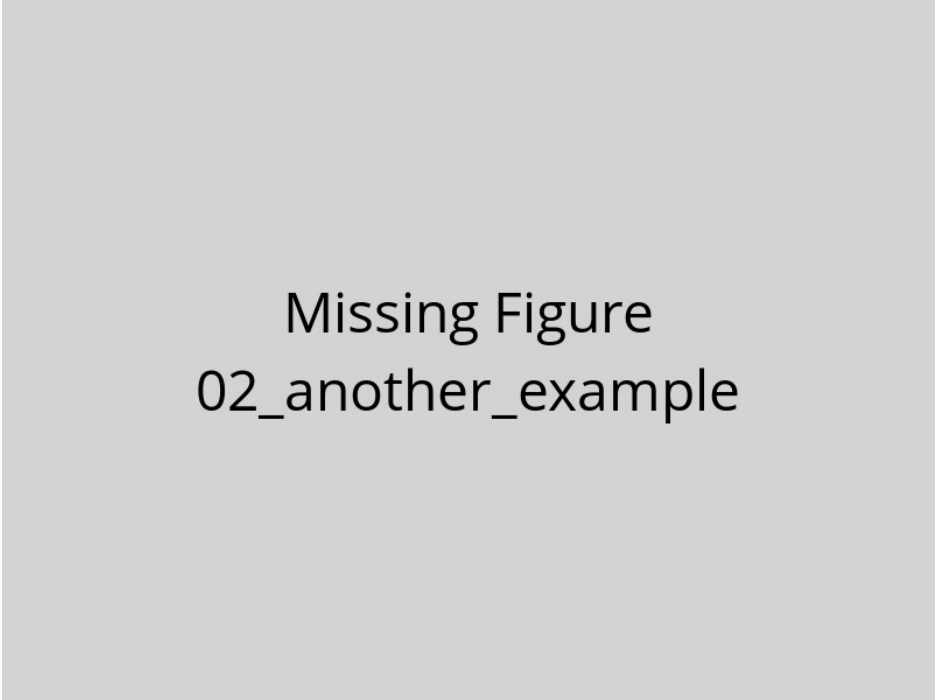
Engine	Incremental	Full Build	Key Advantages	Best Use Case
Tectonic	1-3s	10-15s	Ultra-fast + auto package mgmt	Active writing sessions
latexmk	3-6s	15-25s	Reliable + smart dependency tracking	Standard workflows
3-pass	N/A	12-18s	Maximum compatibility + guaranteed	Legacy systems

Table 3 – Compilation Engine Performance Characteristics. Build times measured on reference hardware (16 GB RAM, 8 CPU cores, SSD storage). Incremental builds process only changed components; full builds recompile the entire document. Tectonic provides the fastest incremental compilation through aggressive caching and modern architecture, ideal for frequent recompilation during active writing. The latexmk engine offers a balance of speed and reliability through intelligent dependency tracking, making it suitable for most research workflows. Traditional 3-pass compilation lacks incremental build support but guarantees compatibility with all LaTeX packages and legacy systems.



Missing Figure
01_example_figure

Figure 1 – Example figure caption. This is a template showing how to include figures in your manuscript. Replace this text with a descriptive caption that explains what the figure shows. Include panel labels (A, B, C) if using multi-panel figures. Explain abbreviations and symbols used in the figure. Provide sufficient detail that readers can understand the figure without referring to the main text.



Missing Figure
02_another_example

Figure 2 – Another example figure. Use this template to add additional figures to your manuscript. Each figure should be placed in a separate .tex file in this directory. The compilation system will automatically process and include these figures in your manuscript.

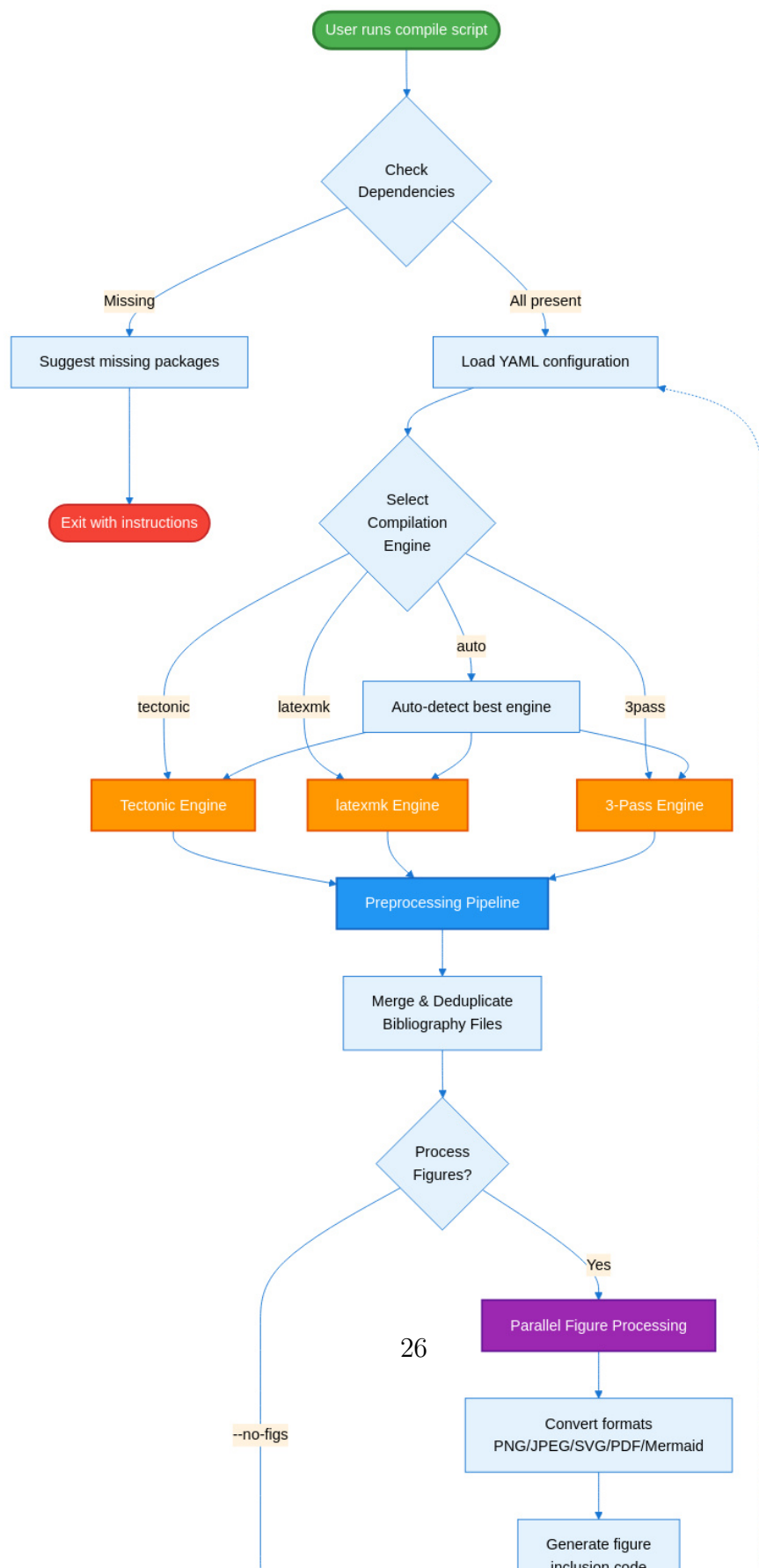




Figure 4 – SciTeX Writer Directory Organization. This diagram shows the hierarchical organization of the repository structure. The `00_shared/` directory (green) contains resources used across all document types, including bibliography files and LaTeX styles, ensuring consistency without duplication. The three document directories (blue/purple) follow identical internal structures, facilitating familiarity and code reuse. Each document has its own `contents/` subdirectory with `figures/` and `tables/` organized into `caption_and_media/` (source files) and `compiled/` (generated LaTeX code). The modular structure minimizes merge conflicts during collaborative editing by isolating frequently-modified content files. Compilation scripts (orange) and configuration files (red) reside in dedicated directories for easy discovery and maintenance. Dotted lines indicate that supplementary materials follow the same organizational pattern as the main manuscript.

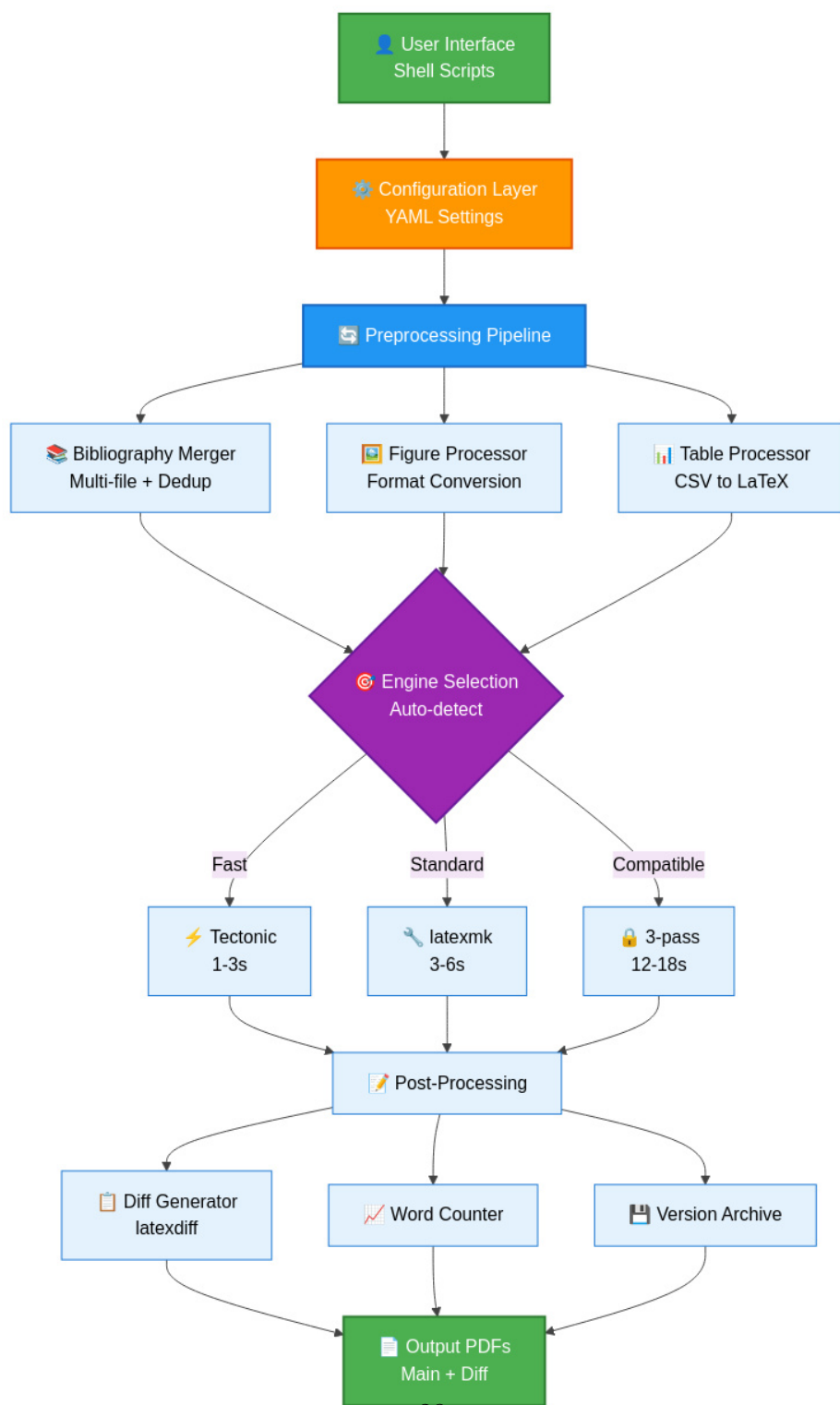


Figure 5 – SciTeX Writer System Architecture. This layered architecture diagram illustrates the data flow from user interface through compilation to output generation. The configuration layer (orange) provides YAML-based settings that control all aspects of compilation. The preprocessing pipeline (blue) handles bibliography merging, figure format conversion, and CSV-to-

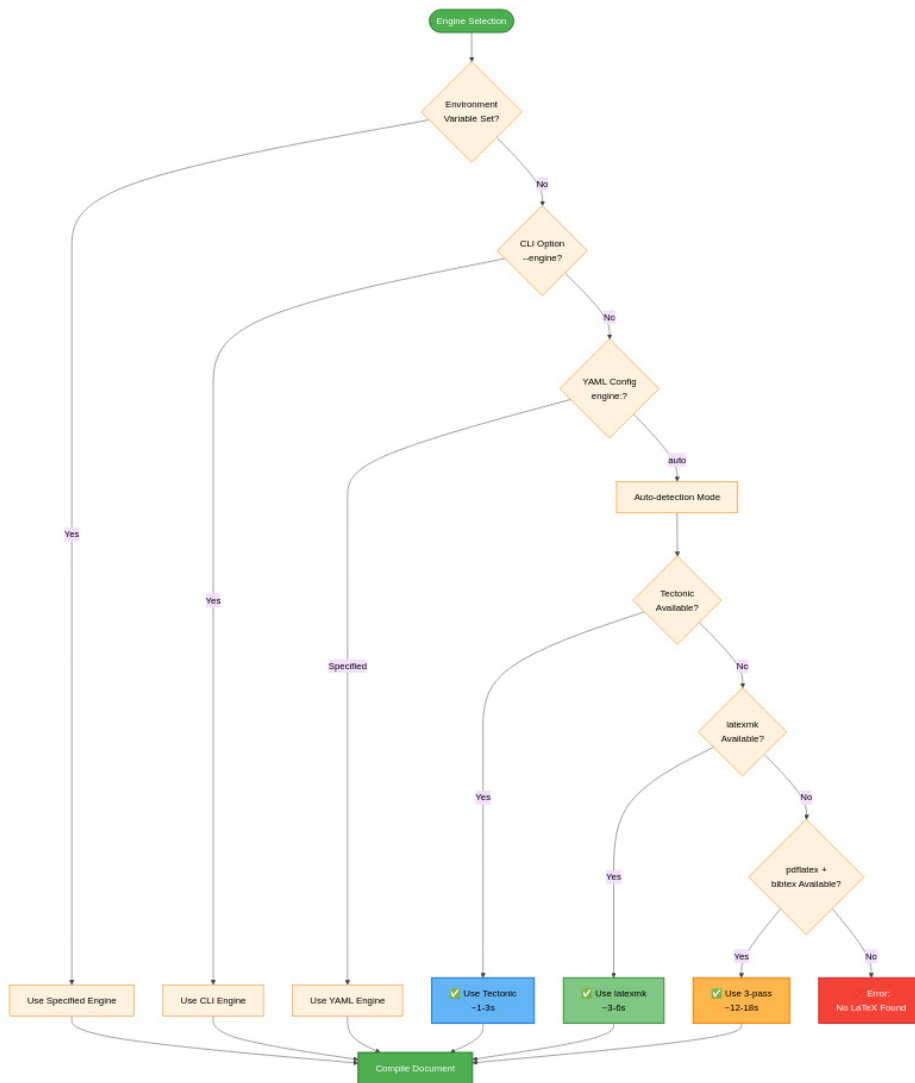


Figure 6 – Compilation Engine Selection Decision Tree. The system prioritizes explicit user configuration over automatic detection. Environment variables provide the highest priority override (useful for CI/CD pipelines). Command-line arguments (`--engine`) override YAML configuration (useful for one-off compilations). When engine is set to 'auto' (default), the system attempts to use Tectonic first for speed, falling back to latexmk for reliability, and finally to 3-pass for maximum compatibility. This cascading detection ensures the system always finds an available compilation method while preferring faster engines when possible. Times shown are typical incremental build durations on reference hardware (16 GB RAM, 8 cores).

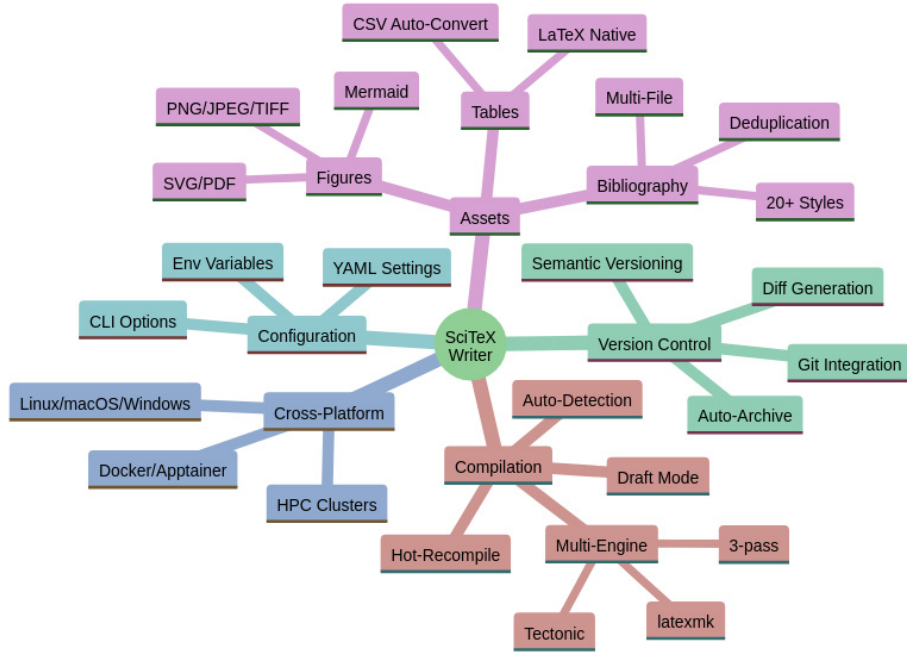


Figure 7 – SciTeX Writer Feature Overview. This mind map provides a comprehensive visualization of the framework’s major capabilities organized into five categories. Compilation features include multi-engine support with auto-detection and performance optimization modes. Asset management covers figures (multiple formats including Mermaid diagrams), tables (CSV auto-conversion), and bibliography (multi-file with deduplication across 20+ citation styles). Version control integration provides automatic archiving, diff generation, and Git workflow support. Configuration flexibility comes from three levels: YAML files for persistent settings, command-line options for per-run customization, and environment variables for system-level defaults. Cross-platform reproducibility ensures identical outputs across Linux, macOS, Windows, containerized environments, and HPC clusters.



Figure 8 – Multi-File Bibliography Processing and Deduplication.

The bibliography system enables researchers to organize references across multiple topical .bib files. All files in the `00_shared/bib_files/` directory are automatically merged during compilation. The deduplication engine implements a two-tier matching strategy: DOI-based matching provides exact duplicate detection when DOIs are present, while title and year matching handles entries without DOIs. This approach eliminates duplicate references that commonly appear when the same paper is cited across multiple source files. The final bibliography is sorted according to the selected citation style (by citation order for numbered styles, alphabetically for author-year styles). Color coding: blue (merging), purple (deduplication logic), green (output).